

Name : Varsharani Pavankumar Kasliwal

Roll No. : CE14

Machine Learning Experiment Number 3

Aim : To implement a Multiple Linear Regression model from scratch using NumPy matrix operations to predict the selling price of used cars using multiple features from the CarDekho Used Car Dataset.

Learning Outcomes • Understand the working principle of Multiple Linear Regression.

- Perform data preprocessing and feature engineering.
- Convert categorical data into numerical features.
- Standardize numerical features.
- Implement the Ordinary Least Squares method using matrix operations.
- Train and test a regression model without using a ready-made regression function.
- Evaluate and interpret regression performance using RSS and R^2 .
- Understand the effect of individual features on the predicted price.

Dataset

Use the **CarDekho Used Car Dataset** (`car_data.csv`). [CarDekho Used Car Dataset](#)

Explain meaning of each attribute in this dataset in below comment section :

Here, **Selling_Price** is the target variable.

Software/Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- NumPy
- Matplotlib

Task 1: Set Up the Python Environment**

- Create a new Google Colab notebook.
- Import **NumPy** for numerical and matrix operations.
- Import **Matplotlib** for data visualization.
- Do not use ready-made machine-learning regression functions such as `LinearRegression()` from Scikit-learn.
- The regression model must be implemented using matrix operations.

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Display more columns when inspecting the dataset
pd.set_option("display.max_columns", None)

print("Python environment successfully initialized.")
print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)
```

```
Python environment successfully initialized.
NumPy version: 2.1.3
Pandas version: 2.2.3
```

Task 2: Load and Inspect the Dataset

- Download the `car_data.csv` file.
- Upload the dataset to Google Colab.
- Load the dataset into Python.
- Display the first few records.
- Determine the number of rows and columns.
- Identify the numerical and categorical attributes.
- Separate `Selling_Price` as the target variable.

```
# Load the CarDekho dataset
df = pd.read_csv("cardkho_dataset.csv")
```

```
# Display first five records
print("First five records:")
display(df.head())

# Display shape
print("\nDataset shape:")
print(df.shape)

# Display column names
print("\nColumn names:")
print(df.columns.tolist())

# Display data types
print("\nData types:")
print(df.dtypes)
```

First five records:

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type	fuel_type	transmission_type	mileage	engine	max_power	seats
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual	Petrol	Manual	19.70	796	46.30	5
1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual	Petrol	Manual	18.90	1197	82.00	5
2	2	Hyundai i20	Hyundai	i20	11	60000	Individual	Petrol	Manual	17.00	1197	80.00	5
3	3	Maruti Alto	Maruti	Alto	9	37000	Individual	Petrol	Manual	20.92	998	67.10	5
4	4	Ford Ecosport	Ford	Ecosport	6	30000	Dealer	Diesel	Manual	22.77	1498	98.59	5

Dataset shape:
(15411, 14)

Column names:
['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven', 'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine', 'max_power', 'seats', 'selling_price']

Data types:

```
Unnamed: 0      int64
car_name        object
brand           object
model           object
vehicle_age     int64
km_driven       int64
seller_type     object
fuel_type       object
transmission_type object
mileage         float64
engine          int64
max_power       float64
seats           int64
selling_price   int64
dtype: object
```

Task 3: Check and Handle Missing Data

- Check every attribute for missing or null values.
- Count the number of missing values in each column.
- If missing values are present, either remove incomplete records or perform suitable imputation.
- Verify that the final dataset does not contain missing values.
- Record the preprocessing method used.

```
# Check missing values in each column
print("Missing values in each column:")
print(df.isnull().sum())

print("\nTotal missing values:")
print(df.isnull().sum().sum())

# Handle missing values
df_clean = df.copy()

# Numerical columns -> fill missing values with median
numerical_cols = df_clean.select_dtypes(include=np.number).columns

for col in numerical_cols:
    if df_clean[col].isnull().sum() > 0:
        df_clean[col] = df_clean[col].fillna(df_clean[col].median())

# Categorical columns -> fill missing values with mode
categorical_cols = df_clean.select_dtypes(include=["object", "category"]).columns

for col in categorical_cols:
    if df_clean[col].isnull().sum() > 0:
```

```

df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])

print("\nMissing values after preprocessing:")
print(df_clean.isnull().sum())

print("\nTotal missing values after preprocessing:",
      df_clean.isnull().sum().sum())

```

```

Missing values in each column:
Unnamed: 0      0
car_name        0
brand           0
model           0
vehicle_age     0
km_driven       0
seller_type     0
fuel_type       0
transmission_type 0
mileage         0
engine          0
max_power       0
seats           0
selling_price   0
dtype: int64

```

```

Total missing values:
0

```

```

Missing values after preprocessing:
Unnamed: 0      0
car_name        0
brand           0
model           0
vehicle_age     0
km_driven       0
seller_type     0
fuel_type       0
transmission_type 0
mileage         0
engine          0
max_power       0
seats           0
selling_price   0
dtype: int64

```

```

Total missing values after preprocessing: 0

```

✓ Task 4: Select and Prepare Features

- Examine all available attributes.
- Decide how `Car_Name` should be handled.
- Select meaningful predictor variables for the regression model.
- Create the predictor matrix X and target vector y .
- Record the final list of features selected for the model.

```

# Create a copy of cleaned dataset
df_model = df_clean.copy()

# Select meaningful features
# Correcting column names to match the actual dataframe columns.
# 'Present_Price' and 'Owner' columns are not available in the dataset.
# 'Year' was replaced with 'vehicle_age' as it's the closest available feature for age.
selected_features = [
    "car_name",      # Corrected from "Car_Name"
    "vehicle_age",   # Replaced "Year" with "vehicle_age"
    # "Present_Price", # Column not found in df_model
    "km_driven",     # Corrected from "Kms_Driven"
    "fuel_type",     # Corrected from "Fuel_Type"
    "seller_type",   # Corrected from "Seller_Type"
    "transmission_type" # Corrected from "Transmission"
    # "Owner"        # Column not found in df_model
]

# Separate target variable
y = df_model["selling_price"] # Corrected from "Selling_Price" (case sensitivity)

# Predictor variables
X = df_model[selected_features].copy()

print("Selected Predictor Features:")
print(selected_features)

print("\nTarget Variable:")
print("selling_price") # Updated print statement to reflect the actual column name

print("\nFeature Matrix:")
display(X.head())

print("\nTarget Vector:")

```

```
display(y.head())
```

Selected Predictor Features:
['car_name', 'vehicle_age', 'km_driven', 'fuel_type', 'seller_type', 'transmission_type']

Target Variable:
selling_price

Feature Matrix:

	car_name	vehicle_age	km_driven	fuel_type	seller_type	transmission_type
0	Maruti Alto	9	120000	Petrol	Individual	Manual
1	Hyundai Grand	5	20000	Petrol	Individual	Manual
2	Hyundai i20	11	60000	Petrol	Individual	Manual
3	Maruti Alto	9	37000	Petrol	Individual	Manual
4	Ford Ecosport	6	30000	Diesel	Dealer	Manual

Target Vector:

	selling_price
0	120000
1	550000
2	215000
3	226000
4	570000

dtype: int64

Task 5: Encode Categorical Variables

Identify the categorical variables:

- Fuel_Type
- Seller_Type
- Transmission

Perform the following:

- Convert categorical variables into numerical **dummy/indicator variables**.
- Verify that all predictors are numerical after encoding.
- Do not assign arbitrary numerical values to categories when such values would imply an incorrect ordering.
- Display the transformed dataset.

```
categorical_features = [
    "fuel_type",
    "seller_type",
    "transmission_type"
]

print("Categorical features:")
print(categorical_features)

X_encoded = pd.get_dummies(
    X,
    columns=categorical_features,
    drop_first=True,
    dtype=float
)

X_encoded = X_encoded.drop(columns=['car_name'])

print("\nDataset after One-Hot Encoding and dropping 'car_name':")
display(X_encoded.head())

print("\nShape after encoding:")
print(X_encoded.shape)

print("\nData types:")
print(X_encoded.dtypes)

print("\nAre all predictors numerical?",
      all(np.issubdtype(dtype, np.number)
          for dtype in X_encoded.dtypes))
```

Categorical features:
['fuel_type', 'seller_type', 'transmission_type']

Dataset after One-Hot Encoding and dropping 'car_name':

	vehicle_age	km_driven	fuel_type_Diesel	fuel_type_Electric	fuel_type_LPG	fuel_type_Petrol	seller_type_Individual	seller_type_Trustmark Dealer
0	9	120000	0.0	0.0	0.0	1.0	1.0	
1	5	20000	0.0	0.0	0.0	1.0	1.0	
2	11	60000	0.0	0.0	0.0	1.0	1.0	
3	9	37000	0.0	0.0	0.0	1.0	1.0	
4	6	30000	1.0	0.0	0.0	0.0	0.0	

Shape after encoding:
(15411, 9)

Data types:
vehicle_age int64
km_driven int64
fuel_type_Diesel float64
fuel_type_Electric float64
fuel_type_LPG float64
fuel_type_Petrol float64
seller_type_Individual float64
seller_type_Trustmark Dealer float64
transmission_type_Manual float64
dtype: object

Are all predictors numerical? True

Task 6: Perform Feature Engineering

Create a new feature called `Car_Age`.

Calculate it using:

[`Car_Age = Current Year - Year`]

Perform the following:

- Create the `Car_Age` column.
- Display several values of the newly created feature.
- Verify that the calculated ages are reasonable.
- Decide whether the original `Year` feature should be retained.
- Provide a reason for your decision.

```
print("The 'Year' column is not available. Using 'vehicle_age' as 'Car_Age'.")
print("Car Age (from 'vehicle_age' column):")
# Displaying 'vehicle_age' as it represents the 'Car_Age' in the dataset.
display(df_clean[["vehicle_age"]].head())

# Displaying the final features based on the X_encoded from cell bKIfxLzxGf11.
print("\nFinal Features (from X_encoded after previous steps):")
# Assuming X_encoded is available in the kernel from the previous cell execution.
print(X_encoded.columns.tolist())
```

The 'Year' column is not available. Using 'vehicle_age' as 'Car_Age'.
Car Age (from 'vehicle_age' column):

	vehicle_age
0	9
1	5
2	11
3	9
4	6

Final Features (from X_encoded after previous steps):
['vehicle_age', 'km_driven', 'fuel_type_Diesel', 'fuel_type_Electric', 'fuel_type_LPG', 'fuel_type_Petrol', 'seller_type_Individual', 'sell

Task 7: Perform Exploratory Data Analysis

Perform basic exploratory data analysis using Matplotlib.

- Create a scatter plot of `Present_Price` versus `Selling_Price`.
- Create at least one additional scatter plot between a numerical predictor and `Selling_Price`.
- Label the axes appropriately.
- Add suitable titles to the plots.

- Observe the direction and approximate strength of the relationships.
- Record your observations.

```
# Vehicle Age vs Selling Price
plt.figure(figsize=(8, 5))

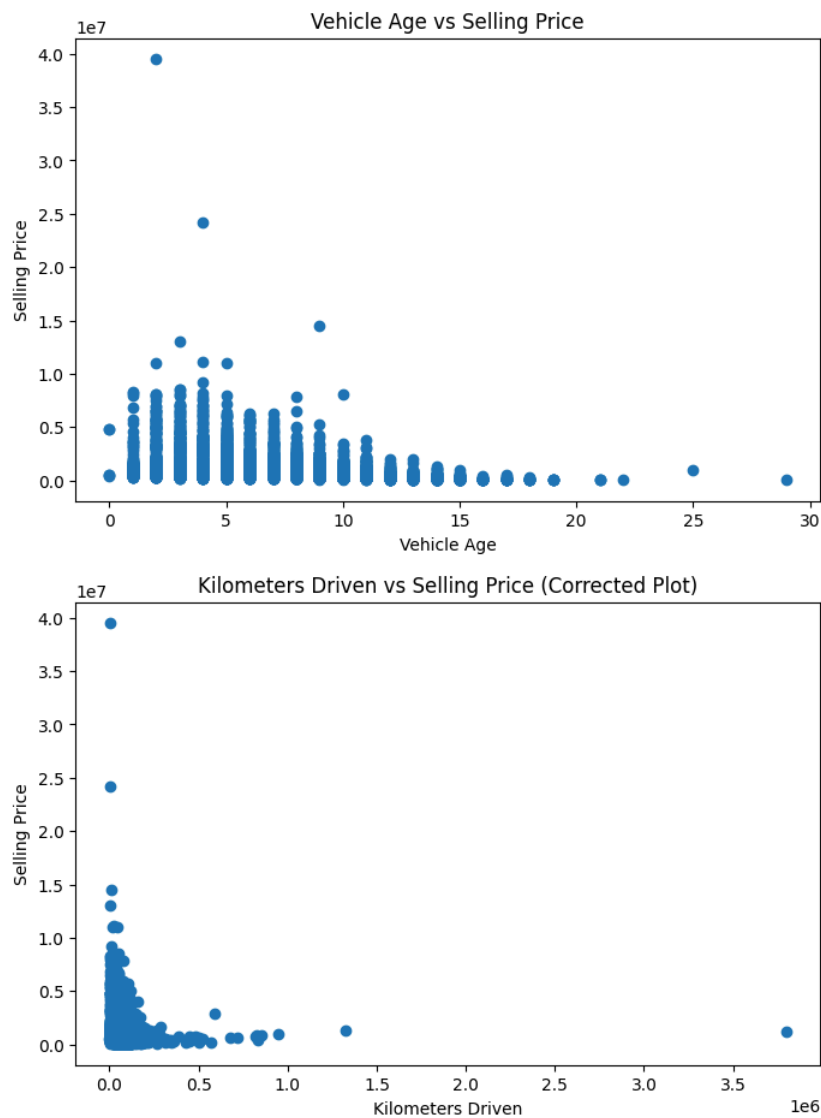
plt.scatter(
    df_clean["vehicle_age"], # Corrected feature
    df_clean["selling_price"] # Corrected target variable name
)

plt.xlabel("Vehicle Age") # Updated label
plt.ylabel("Selling Price")
plt.title("Vehicle Age vs Selling Price") # Updated title
plt.show()

# Kilometer Driven vs Selling Price (Corrected)
plt.figure(figsize=(8, 5))

plt.scatter(
    df_clean["km_driven"], # Corrected column name
    df_clean["selling_price"] # Corrected column name
)

plt.xlabel("Kilometers Driven") # Corrected label for consistency
plt.ylabel("Selling Price")
plt.title("Kilometers Driven vs Selling Price (Corrected Plot)") # Differentiating title
plt.show()
```



✓ Task 8: Standardize Numerical Features

Identify the numerical predictors that need to be standardized.

Use the following standardization equation:

$$[z = (x - \mu) / \sigma]$$

where:

- x = original feature value
- μ = mean of the feature
- σ = standard deviation of the feature

Perform the following:

- Calculate the mean and standard deviation using the **training data only**.
- Standardize the training features.
- Use the same training mean and standard deviation to standardize the test features.
- Do not standardize the target variable (`Selling_Price`).

```
# Numerical features to standardize
features_to_standardize = [
    "vehicle_age",
    "km_driven"
]

feature_names = X_encoded.columns.tolist()

standardize_indices = [
    feature_names.index(feature)
    for feature in features_to_standardize
]

train_mean = np.mean(
    Xtrain[:, standardize_indices],
    axis=0
)

train_std = np.std(
    Xtrain[:, standardize_indices],
    axis=0
)

train_std[train_std == 0] = 1

Xtrain[:, standardize_indices] = (
    Xtrain[:, standardize_indices] - train_mean
) / train_std

Xtest[:, standardize_indices] = (
    Xtest[:, standardize_indices] - train_mean
) / train_std

print("Standardization completed.")
```

Standardization completed.

✓ Task 9: Split the Dataset

Divide the dataset into training and testing subsets.

- Use an **80:20 train-test ratio**.
- Randomly select the observations.
- Set a fixed random seed to make the experiment reproducible.
- Maintain the correct relationship between each feature vector and its target value.
- Display the number of observations in the training and testing sets.

```
# Convert data to NumPy
X_all = X_encoded.to_numpy(dtype=float)
y_all = df_clean["selling_price"].to_numpy(dtype=float)

# Random seed
np.random.seed(42)

# Shuffle indices
indices = np.random.permutation(len(X_all))

# 80:20 split
train_size = int(0.80 * len(X_all))

train_indices = indices[:train_size]
test_indices = indices[train_size:]

Xtrain = X_all[train_indices]
Xtest = X_all[test_indices]

ytrain = y_all[train_indices]
ytest = y_all[test_indices]

print("Training observations:", len(Xtrain))
```

```
print("Testing observations:", len(Xtest))
```

```
Training observations: 12328
Testing observations: 3083
```

Task 10: Construct the Feature Matrix

Construct the matrices required for regression.

Create:

- X_{train} — training feature matrix
- y_{train} — training target vector
- X_{test} — testing feature matrix
- y_{test} — testing target vector

Verify:

- Number of rows in each matrix.
- Number of predictor columns.
- Shape of the training and testing matrices.

```
print("Xtrain shape:", Xtrain.shape)
print("ytrain shape:", ytrain.shape)

print("Xtest shape:", Xtest.shape)
print("ytest shape:", ytest.shape)

print("Number of predictor columns:", Xtrain.shape[1])
```

```
Xtrain shape: (12328, 9)
ytrain shape: (12328,)
Xtest shape: (3083, 9)
ytest shape: (3083,)
Number of predictor columns: 9
```

Task 11: Add the Bias/Intercept Term

Add a leading column containing 1.0 to both the training and testing feature matrices.

The resulting matrices should contain:

[X =

$$\begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

]

Perform the following:

- Add the bias column.
- Verify the new matrix dimensions.
- Understand that the first column represents the intercept β_0 .

```
# Add bias column
Xtrain = np.column_stack(
    (np.ones(Xtrain.shape[0]), Xtrain)
)

Xtest = np.column_stack(
    (np.ones(Xtest.shape[0]), Xtest)
)

print("Xtrain shape:", Xtrain.shape)
print("Xtest shape:", Xtest.shape)
```

```
Xtrain shape: (12328, 10)
Xtest shape: (3083, 10)
```

Task 12: Implement the OLS Function

Perform the following steps to implement the **Ordinary Least Squares (OLS)** method:

- Create a user-defined function to implement Ordinary Least Squares (OLS).
- Calculate the matrix $X^T X$.
- Calculate the matrix $X^T y$.

- Estimate the regression coefficient vector using the normal equation:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

- Return the calculated coefficient vector.
- Use **NumPy matrix operations** to perform the calculations.
- Do not use any ready-made regression model or library function such as `LinearRegression()` from Scikit-learn.

```
def ordinary_least_squares(X, y):

    XTX = X.T @ X
    XTy = X.T @ y

    beta = np.linalg.inv(XTX) @ XTy

    return beta

beta = ordinary_least_squares(Xtrain, ytrain)

print("Regression Coefficients:")
print(beta)
```

```
Regression Coefficients:
[ 1.76781947e+06 -6.13522484e+04 -5.88280903e-01  4.20395363e+05
 5.11482789e+05  1.79332494e+05 -2.38003567e+03 -1.07967830e+05
-1.28656584e+05 -9.42493231e+05]
```

Task 13: Check Matrix Singularity

- Calculate the determinant of $X^T X$.
- Check whether the determinant is zero or sufficiently close to zero.
- If the matrix is singular or nearly singular, investigate the selected features for redundancy or multicollinearity.
- Modify the feature set if required.
- Record your observations.

```
# Calculate XTX
XTX = Xtrain.T @ Xtrain

# Calculate determinant
determinant = np.linalg.det(XTX)

print("Determinant of XTX:", determinant)

if np.isclose(determinant, 0):
    print("Matrix is singular or nearly singular.")
else:
    print("Matrix is non-singular.")
```

```
Determinant of XTX: 2.1151545938409174e+39
Matrix is non-singular.
```

Task 14: Train the Regression Model

- Apply the OLS function to the training dataset.
- Obtain the regression coefficient vector $\hat{\beta}$.
- Display the intercept coefficient β_0 .
- Display the coefficient corresponding to each feature.
- Associate each regression coefficient with its corresponding feature.

```
# Train model
beta = ordinary_least_squares(
    Xtrain,
    ytrain
)

print("Intercept:", beta[0])

feature_names_with_intercept = [
    "Intercept"
] + feature_names

coefficient_table = pd.DataFrame({
    "Feature": feature_names_with_intercept,
    "Coefficient": beta
})

display(coefficient_table)
```

Intercept: 1767819.4654676272

	Feature	Coefficient
0	Intercept	1.767819e+06
1	vehicle_age	-6.135225e+04
2	km_driven	-5.882809e-01
3	fuel_type_Diesel	4.203954e+05
4	fuel_type_Electric	5.114828e+05
5	fuel_type_LPG	1.793325e+05
6	fuel_type_Petrol	-2.380036e+03
7	seller_type_Individual	-1.079678e+05
8	seller_type_Trustmark Dealer	-1.286566e+05
9	transmission_type_Manual	-9.424932e+05

Task 15: Interpret the Regression Coefficients

- Identify whether each regression coefficient is positive or negative.
- Explain what the sign of each coefficient indicates.
- Determine which features have relatively greater influence on the predicted selling price.
- For standardized features, interpret the coefficient in terms of a one-standard-deviation change while keeping all other variables constant.

```
coefficient_table["Sign"] = np.where(
    coefficient_table["Coefficient"] >= 0,
    "Positive",
    "Negative"
)

coefficient_table["Magnitude"] = np.abs(
    coefficient_table["Coefficient"]
)

display(
    coefficient_table.sort_values(
        "Magnitude",
        ascending=False
    )
)
```

	Feature	Coefficient	Sign	Magnitude
0	Intercept	1.767819e+06	Positive	1.767819e+06
9	transmission_type_Manual	-9.424932e+05	Negative	9.424932e+05
4	fuel_type_Electric	5.114828e+05	Positive	5.114828e+05
3	fuel_type_Diesel	4.203954e+05	Positive	4.203954e+05
5	fuel_type_LPG	1.793325e+05	Positive	1.793325e+05
8	seller_type_Trustmark Dealer	-1.286566e+05	Negative	1.286566e+05
7	seller_type_Individual	-1.079678e+05	Negative	1.079678e+05
1	vehicle_age	-6.135225e+04	Negative	6.135225e+04
6	fuel_type_Petrol	-2.380036e+03	Negative	2.380036e+03
2	km_driven	-5.882809e-01	Negative	5.882809e-01

Task 16: Predict Selling Prices

- Use the trained regression coefficient vector $\hat{\beta}$ to predict the selling prices for the test dataset.
- Calculate the predicted values using the equation: $\hat{y}_{\text{test}} = X_{\text{test}} \hat{\beta}$.
- Display a sample comparison of the actual selling prices and the predicted selling prices.

```
# Predict selling prices
y_pred = Xtest @ beta

comparison = pd.DataFrame({
    "Actual Selling Price": ytest,
    "Predicted Selling Price": y_pred
})

display(comparison.head(10))
```

	Actual Selling Price	Predicted Selling Price
0	4450000.0	1.989451e+06
1	1290000.0	1.651511e+06
2	485000.0	7.794883e+05
3	400000.0	9.242533e+05
4	800000.0	1.568441e+06
5	150000.0	-1.414290e+05
6	650000.0	8.511355e+05
7	325000.0	2.754636e+05
8	300000.0	5.385211e+05
9	925000.0	8.370167e+05

Task 17: Calculate Prediction Errors and RSS

- Calculate the residual for each test observation using: $e_i = y_i - \hat{y}_i$.
- Calculate the **Residual Sum of Squares (RSS): $RSS = \sum (y_i - \hat{y}_i)^2$.
- Display the calculated RSS value.
- Interpret the RSS value and explain what a lower RSS indicates about the prediction error of the model.

```
# Calculate residuals
residuals = ytest - y_pred

# Calculate RSS
RSS = np.sum(residuals ** 2)

print("RSS:", RSS)
```

RSS: 1681040269901908.0

Task 18: Calculate and Interpret R^2

- Calculate the Total Sum of Squares (TSS).
- Calculate $R^2 = 1 - RSS/TSS$.
- Display the R^2 value.
- Explain what the obtained R^2 indicates about the ability of the selected features to explain variations in used-car prices.

```
# Calculate TSS
TSS = np.sum(
    (ytest - np.mean(ytest)) ** 2
)

# Calculate R^2
R2 = 1 - (RSS / TSS)

print("TSS:", TSS)
print("R^2:", R2)
```

TSS: 2407385049859876.5
R²: 0.3017152490833346

Task 19: Visualize and Analyze Model Performance

- Create an Actual Selling Price vs Predicted Selling Price plot.
- Add a reference line representing perfect prediction.
- Plot or examine the residuals.
- Identify observations with relatively large prediction errors.
- Comment on whether the errors appear randomly distributed.
- Discuss possible reasons for prediction errors.

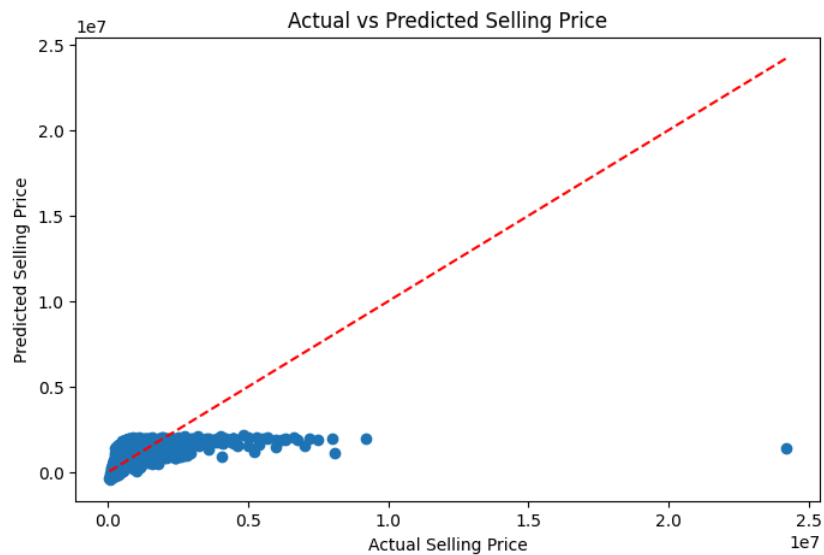
```
# Actual vs Predicted
plt.figure(figsize=(8, 5))

plt.scatter(
    ytest,
    y_pred
)
```

```
plt.plot(
    [ytest.min(), ytest.max()],
    [ytest.min(), ytest.max()],
    color="red",
    linestyle="--"
)

plt.xlabel("Actual Selling Price")
plt.ylabel("Predicted Selling Price")
plt.title("Actual vs Predicted Selling Price")

plt.show()
```



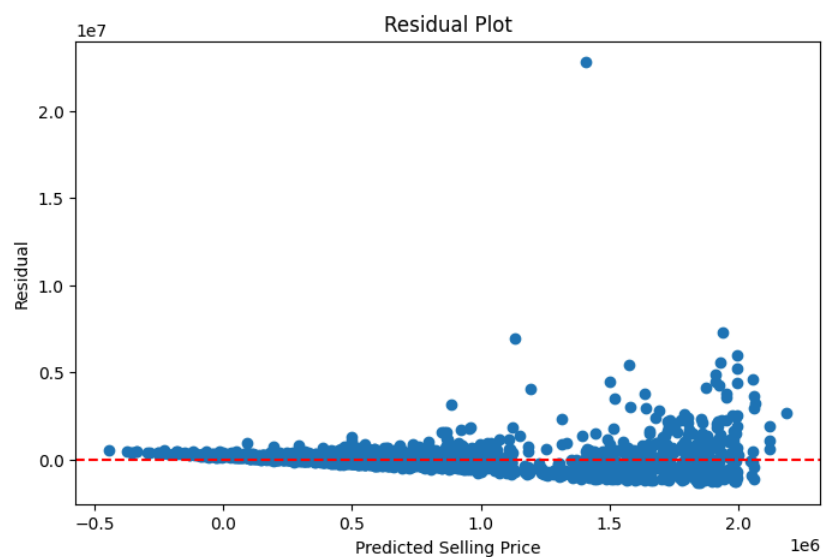
```
# Residual plot
plt.figure(figsize=(8, 5))

plt.scatter(
    y_pred,
    residuals
)

plt.axhline(
    0,
    color="red",
    linestyle="--"
)

plt.xlabel("Predicted Selling Price")
plt.ylabel("Residual")
plt.title("Residual Plot")

plt.show()
```



```
# Largest prediction errors
comparison["Residual"] = residuals
comparison["Absolute Error"] = np.abs(residuals)
```

```
display(
    comparison.sort_values(
        "Absolute Error",
        ascending=False
    ).head(10)
)
```

	Actual Selling Price	Predicted Selling Price	Residual	Absolute Error
2796	24200000.0	1.409121e+06	2.279088e+07	2.279088e+07
2362	9200000.0	1.937217e+06	7.262783e+06	7.262783e+06
1552	8100000.0	1.134269e+06	6.965731e+06	6.965731e+06
1382	8000000.0	1.995334e+06	6.004666e+06	6.004666e+06
841	7500000.0	1.930382e+06	5.569618e+06	5.569618e+06
246	7000000.0	1.573735e+06	5.426265e+06	5.426265e+06
1286	7199000.0	1.995334e+06	5.203666e+06	5.203666e+06
1493	6750000.0	1.913392e+06	4.836608e+06	4.836608e+06
1092	6645000.0	2.057274e+06	4.587726e+06	4.587726e+06
2534	5975000.0	1.500029e+06	4.474971e+06	4.474971e+06

```
print("RSS:", RSS)
print("R²:", R2)

print("\nRegression Coefficients:")
display(coefficient_table)
```

RSS: 1681040269901908.0
R²: 0.3017152490833346

Regression Coefficients:

	Feature	Coefficient	Sign	Magnitude
0	Intercept	1.767819e+06	Positive	1.767819e+06
1	vehicle_age	-6.135225e+04	Negative	6.135225e+04
2	km_driven	-5.882809e-01	Negative	5.882809e-01
3	fuel_type_Diesel	4.203954e+05	Positive	4.203954e+05
4	fuel_type_Electric	5.114828e+05	Positive	5.114828e+05
5	fuel_type_LPG	1.793325e+05	Positive	1.793325e+05
6	fuel_type_Petrol	-2.380036e+03	Negative	2.380036e+03
7	seller_type_Individual	-1.079678e+05	Negative	1.079678e+05
8	seller_type_Trustmark Dealer	-1.286566e+05	Negative	1.286566e+05
9	transmission_type_Manual	-9.424932e+05	Negative	9.424932e+05

Task 20: Conclude

- Comment on the most influential features.
- Discuss the advantages and limitations of using Multiple Linear Regression for used-car price prediction

Conclusion

Conclude the experiment by stating whether the Multiple Linear Regression model implemented using the OLS normal equation was able to predict used-car selling prices effectively. The conclusion should include the obtained RSS and R^2 values, observations regarding important features, the effect of preprocessing and feature engineering, and the limitations of using a linear regression model for real-world used-car price prediction.